



UNIVERSITÀ
DI TRENTO

Prototipazione di un'interfaccia georeferenziata basata su SoC RP2040

Selmir Kusi, Mattia Fait, Alessandro Biasioli
Corso in Progettazione e Prototipazione di Sistemi Elettronici (DISI)

Anno Accademico 2025/2026

Sommario

La presente relazione espone la **prototipazione, programmazione e validazione** di un'unità di calcolo *embedded* custom basata sul SoC *Raspberry Pi RP2040*, nonché dotata di sensoristica per il *data-logging* georeferenziato in contesti IoT. L'architettura proposta sfrutta il determinismo delle unità di I/O programmabili (*Programmable I/O* - PIO) per l'offloading hardware di task critici di timing, preservando la reattività dei core *ARM Cortex-M0+* durante la gestione delle periferiche di comunicazione. Particolare attenzione è rivolta alla robustezza del sistema, garantita dall'impiego di un *filesystem fail-safe* (*LittleFS*) su memoria flash QSPI esterna al SoC e dall'adozione di rigorosi standard di manifattura SMT (*Surface Mount Technology*). Questo prototipo dimostra l'efficacia dell'integrazione tra sensori ambientali, moduli GNSS, e firmware, confermando la resilienza dell'architettura in scenari critici per l'acquisizione dati.

Indice

1	Introduzione	2
1.1	Contesto e Scenario Applicativo	2
1.2	Motivazioni Progettuali	2
1.3	Obiettivi del Lavoro	2
2	Sotto-sistema Hardware	3
2.1	Implementazione Fisica e Tecniche di Assemblaggio SMT	3
2.2	Il SoC Raspberry Pi RP2040	3
3	Sotto-sistema Software	6
3.1	Organizzazione del Codice Sorgente e Architettura dei File	6
3.2	Toolchain e Build System	6
3.3	PlatformIO e l'Ecosistema RP2040	8
3.4	Conflitti tra Arduino, pico-sdk e Librerie Esterne e relative soluzioni	10
3.5	Ottimizzazioni di co-design software-hardware	11
3.6	Periferiche e Persistenza in memoria	13
3.7	Sistema di Telemetria GPS	14
3.8	Architettura Multi-Core: organizzazione software	15
3.9	Scheduler dei Task e Cooperazione Multitasking	16
3.10	Sistema di Gestione e Codifica degli Errori	18
4	Interfaccia Utente	20
4.1	FSM e Contratto view_interface_t	20
4.2	Rendering e Timing OLED	20
4.3	View Implementate	20
5	Interfacciamento PC–Board: API Python e Protocollo Seriale	22
5.1	Architettura del Sistema di Sincronizzazione	22
5.2	Lo Script Python	22
6	Conclusioni e Sviluppi Futuri	25
6.1	Analisi dei Risultati e Validazione Tecnica	25
6.2	Riflessioni sul Design Ibrido	25
6.3	Sviluppi Futuri e Scalabilità	25

1 Introduzione

1.1 Contesto e Scenario Applicativo

Nell'attuale ecosistema dell'*Internet of Things* (IoT), l'acquisizione sinergica di parametri ambientali e coordinate geografiche rappresenta un requisito abilitante per il monitoraggio remoto avanzato e l'integrazione nei processi di *Industria 4.0*.

Il *data logging* georeferenziato consiste nella registrazione di flussi di dati sensoriali indissolubilmente legati a coordinate spaziali e riferimenti temporali precisi, permettendo la ricostruzione analitica di mappe fenomenologiche. La criticità di tali sistemi risiede nella necessità di garantire un sincronismo temporale rigoroso tra il campionamento dei sensori e il dato GNSS (*Global Navigation Satellite System*)¹, preservando l'integrità delle informazioni in memorie non volatili anche in condizioni operative isolate o soggette a instabilità energetica [1].

L'applicazione di tali tecnologie spazia dal monitoraggio di micro-climi agricoli per l'ottimizzazione delle risorse (*Smart Farming*), alla tracciabilità di *asset* logistici in tempo reale, fino alla gestione di nodi sensoriali urbani per la mappatura dell'inquinamento atmosferico e acustico.

1.2 Motivazioni Progettuali

I microcontrollori *general-purpose* convenzionali manifestano spesso colli di bottiglia prestazionali quando chiamati a gestire simultaneamente bus di comunicazione seriale (UART, I2C, SPI) e task computazionali intensivi. Questa saturazione delle risorse introduce *jitter*² indesiderato nei segnali di I/O, degradando la precisione del sistema.

L'adozione del SoC *Raspberry Pi RP2040* [2] risponde alla necessità di superare tali limitazioni attraverso un'architettura *dual-core* simmetrica³ e, soprattutto, mediante l'uso delle unità *Programmable I/O* (PIO) [3]. Queste ultime permettono un approccio di *hardware-software co-design*, dove le macchine a stati dedicate gestiscono il *timing* delle periferiche in modo deterministico, svincolando i core principali dall'onere della gestione dei protocolli a basso livello.

1.3 Obiettivi del Lavoro

Il presente elaborato descrive l'intero processo di prototipazione della *scheda*, analizzandone ogni fase critica: come la validazione del *filesystem fail-safe LittleFS*⁴ per la prevenzione della corruzione dei dati.

L'obiettivo primario è la realizzazione di una piattaforma *embedded*, ottimizzata per il *data-logging* geo-referenziato ad alta frequenza. La documentazione di riferimento, le API sviluppate e i dettagli tecnici sull'architettura firmware sono resi disponibili pubblicamente [4–6], unitamente ai report accademici e tecnici di validazione [7, 8].

Le funzionalità principali del sistema comprendono l'acquisizione di dati GPS (modulo SAM-M8Q), la sincronizzazione dei relativi dati meteo da un PC host tramite seriale, la lettura di sensori ambientali via I2C, l'interazione tramite display OLED SSD1306, e garantire la persistenza su flash. Viene inoltre previsto un feedback visivo su strip WS2812B e acustico tramite buzzer PWM.

¹Il sistema gestisce la ricezione concorrente di costellazioni GPS (USA), GLONASS (Russia) e Galileo (EU) per massimizzare la precisione del fix in ambienti ostili.

²Variazione statistica indesiderata nel ritardo di propagazione di un segnale periodico rispetto a un riferimento temporale ideale.

³L'architettura *ARM Cortex-M0+* del RP2040 permette di dedicare un core al processamento dei dati sensoriali e l'altro alla gestione dello stack di comunicazione o del *filesystem*, riducendo le latenze di *interrupt*.

⁴*LittleFS* è un *filesystem* progettato specificamente per microcontrollori, dotato di meccanismi di *wear-leveling* e resilienza ai crash energetici (*copy-on-write*).

2 Sotto-sistema Hardware

2.1 Implementazione Fisica e Tecniche di Assemblaggio SMT

La costruzione del prototipo ha seguito standard industriali per garantire l'affidabilità meccanica e l'integrità elettrica delle connessioni ad alta frequenza. Il montaggio dei componenti è stato eseguito prevalentemente tramite tecnologia SMT⁵ (*Surface Mount Technology*) e saldatura a rifusione, scelta dettata da precise necessità progettuali:

- **Riduzione delle Induttanze Parassite:** A differenza della tecnologia *Through-Hole*, l'assenza di reofori (pin passanti) lunghi minimizza le induttanze parassite. Questo riduce drasticamente fenomeni di *ringing* e riflessioni del segnale, preservando l'integrità dei fronti d'onda digitali.
- **Ottimizzazione del Piano di Massa:** La tecnologia SMT permette una connessione diretta dei *thermal pad* al piano di massa inferiore attraverso la tecnica dei *via-in-pad*. Questo è fondamentale non solo per la dissipazione termica del SoC, ma anche per offrire un riferimento di tensione estremamente stabile e a bassa impedenza.

Integrazione del modulo GNSS SAM-M8Q

Il modulo GNSS **u-blox SAM-M8Q** [1], in package **LCC** (*Leadless Chip Carrier*), è stato integrato tramite saldatura manuale di precisione. Sebbene il componente sia progettato per processi SMT industriali, le variazioni nel layout di prototipazione hanno reso più efficace l'assemblaggio manuale. Questa procedura è stata condotta sotto controllo ottico per garantire la corretta bagnabilità dei pad perimetrali.

Nonostante l'assenza di un processo di rifusione industriale, l'approccio adottato ha garantito:

1. **Integrità delle Connessioni:** La verifica visiva e strumentale ha escluso la presenza di *solder bridges*, particolarmente critici dato il *pitch* ridotto dei contatti.
2. **Preservazione RF:** La cura riposta nella saldatura dei pad relativi all'antenna integrata ha permesso di mantenere un *Carrier-to-Noise density ratio* (C/N0) operativo, fondamentale per minimizzare le perdite di inserzione e garantire l'aggancio dei satelliti, pur nei limiti strutturali di un prototipo assemblato manualmente.

2.2 Il SoC Raspberry Pi RP2040

Il sistema orbita attorno al SoC **RP2040** [2], caratterizzato da un'architettura dual-core ARM Cortex-M0+ con bus AMBA AHB [2]. La struttura multi-core simmetrica permette di parallelizzare l'acquisizione dai sensori e la scrittura su memoria flash, entrambe fasi critiche e non interrompibili.

Meccanismo di Interrupt vs Polling

La scelta del paradigma di gestione degli input è critica per garantire il determinismo del sistema e la reattività dell'interfaccia utente. Si è scelto di escludere la tecnica del *polling* — ovvero il campionamento ciclico dello stato dei pin all'interno del ciclo principale — poiché risulterebbe intrinsecamente inefficiente. In un sistema che esegue task computazionalmente intensivi o a latenza variabile, come il parsing delle sentenze NMEA e per le operazioni di scrittura su memoria Flash, il polling comporterebbe un elevato

⁵Surface Mount Technology: tecnologia di assemblaggio che prevede il montaggio dei componenti direttamente sulla superficie del PCB.

rischio di *aliasing temporale*, portando alla perdita di eventi di pressione se questi occorrono mentre la CPU è occupata in altre routine.

L'adozione degli **interrupt hardware (IRQ)** sposta la gestione dell'evento dal dominio software a quello hardware, permettendo una gestione asincrona. Al verificarsi di una transizione di stato sul pin GPIO (fronte di discesa), il controller degli interrupt del core ARM forza una sospensione immediata del programma principale per eseguire una *Interrupt Service Routine* (ISR). Questo garantisce una latenza di risposta minima e costante, svincolando la reattività del sistema dal carico computazionale istantaneo.

Architettura degli Interrupt sull'RP2040 L'efficienza di questa implementazione è legata alla specifica architettura del SoC RP2040. Come documentato nel datasheet [2, Sez. 2.19.3], il sistema non assegna un vettore di interrupt dedicato ad ogni singolo pin, ma aggrega le richieste tramite una logica **OR-wired** per ogni banco di memoria GPIO.

Dato che tutti i pulsanti del dispositivo afferiscono allo stesso banco, la logica di gestione è stata strutturata in due fasi logiche:

1. **Segnalazione Hardware:** L'evento sul pin solleva un segnale di interrupt globale verso il processore.
2. **Discriminazione Software:** All'interno della ISR, il software interroga i registri di stato degli interrupt (*Interrupt Status Registers*) per identificare quale specifico GPIO abbia scatenato l'evento. Questa architettura centralizzata permette di ottimizzare l'uso delle risorse, utilizzando un unico handler per gestire l'intero set di input utente.

Condizionamento e Pull-up Per garantire la stabilità dei livelli logici, i pin sono stati configurati in modalità INPUT_PULLUP. Questa impostazione attiva una rete di resistenze interne al SoC (con valori nominali compresi tra 50 kΩ e 80 kΩ). Tale configurazione, lavorando in sinergia con il filtro RC esterno, assicura un pull-up "forte" che previene stati di alta impedenza (*floating*) e mitiga l'effetto di accoppiamenti capacitivi indesiderati, garantendo una lettura pulita anche in ambienti elettricamente rumorosi.

Architettura Dual-Core e Gestione delle Risorse Condivise

Il SoC RP2040 implementa un'architettura **dual-core simmetrica** basata su due processori **ARM Cortex-M0+** identici e indipendenti, ciascuno dotato del proprio NVIC (*Nested Vectored Interrupt Controller*), del proprio set di registri e del proprio stack pointer.

A differenza dei sistemi multi-core con cache coerente (tipici dei microprocessori), i due core dell'RP2040 non possiedono cache dati: l'accesso alla memoria avviene direttamente attraverso il bus fabric, eliminando ogni problematica di *cache coherency* tra i core.

SIO: Single-cycle I/O Per le operazioni più sensibili alla latenza — in particolare il controllo dei GPIO — l'RP2040 affianca al bus AHB (*Advanced High-performance Bus*) un percorso dedicato denominato **SIO** (*Single-cycle I/O*). Si tratta di una periferica privata per ciascun core, mappata sullo stesso indirizzo logico ma fisicamente duplicata, raggiungibile in un singolo ciclo di clock senza dover attraversare l'arbitraggio del bus fabric. Questo design garantisce che operazioni critiche come il *set/clear/toggle* atomico dei pin GPIO non siano soggette a jitter introdotto da contesa del bus.

Comunicazione Inter-Processore Poiché i due core non condividono cache né un meccanismo nativo di messaggistica a livello applicativo, l'RP2040 mette a disposizione, all'interno del blocco SIO, due **FIFO hardware unidirezionali** (4×32 bit per ciascuna direzione), utilizzate come canale per lo scambio di piccoli messaggi o puntatori tra i core.

Ogni core può generare un interrupt verso l'altro alla scrittura/lettura della propria FIFO, permettendo pattern di sincronizzazione a basso overhead e senza dover ricorrere a polling su variabili condivise in SRAM.

Spinlock Hardware Per la protezione di risorse condivise come strutture dati in SRAM o Memory-Mapped I/O, l'RP2040 fornisce **32 spinlock hardware**, anch'essi esposti tramite il blocco SIO.

Questi spinlock costituiscono il meccanismo di sincronizzazione a livello più basso su cui il Pico SDK costruisce astrazioni di livello superiore come mutex e semaphores.

Routing degli Interrupt fra i Core A differenza dello schema centralizzato descritto per il banco GPIO, il sistema di interrupt dell'RP2040 è organizzato in modo che **ciascun core disponga di un proprio NVIC indipendente**, con maschere di abilitazione (registri NVIC_ICER/NVIC_ISER) configurabili separatamente. È quindi possibile abilitare o disabilitare un interrupt su un core senza che questo influenzi l'altro.

Le sorgenti di interrupt periferiche (UART, I2C, ADC, PWM, GPIO, ecc.) sono instradabili verso uno o entrambi i core tramite i registri INTE/INTS del blocco interessato. Nel sistema sviluppato, il *Core 1* è stato dedicato alla gestione del parsing NMEA, mentre il *Core 0* è stato riservato alle operazioni di scrittura su memoria Flash.

3 Sotto-sistema Software

3.1 Organizzazione del Codice Sorgente e Architettura dei File

L'architettura del firmware adotta un modello di sviluppo modulare basato sulla netta separazione tra le interfacce di dichiarazione (file header `.h`) e le rispettive logiche di implementazione (file sorgente `.cpp`). Questa compartmentalizzazione favorisce l'isolamento delle funzionalità, semplifica le procedure di verifica e ottimizza i tempi di compilazione incrementale gestiti da PlatformIO. La documentazione tecnica dettagliata e l'analisi approfondita del codice sono consultabili presso il repository ufficiale del progetto [5].

L'albero strutturale della directory di sviluppo è organizzato secondo lo schema seguente:

```

firm/
├── include/
│   ├── core/          messages.h, system_manager.h, telemetry.h,
│                       storage.h, config.h, error_codes.h
│   ├── drivers/       display_ssd1306.h, inputs.h, sensors_i2c.h,
│                       peripherals.h, radio_uart.h, config_pins.h
│   ├── pipeline/      core0_manager.h, core1_manager.h, tasks.h
│   ├── ui/            ui_manager.h, view_definitions.h, boot_anim.h
│   └── util/          scheduler.h, target.h
├── src/
│   ├── main.cpp
│   ├── core/          system_manager.cpp, telemetry.cpp,
│                       storage.cpp, config.cpp, power_manager.cpp
│   ├── drivers/       display_ssd1306.cpp, inputs.cpp, sensors_i2c.cpp,
│                       peripherals.cpp, radio_uart.cpp
│   ├── pipeline/      core0_manager.cpp, core1_manager.cpp, tasks.cpp
│   ├── ui/            ui_manager.cpp, boot_anim.cpp
│   │   └── views/      view_gps.cpp, view_home.cpp, view_info.cpp,
│                       view_meteo.cpp, view_settings.cpp
└── lib/
    └── minmea/         minmea.c, minmea.h
  
```

3.2 Toolchain e Build System

Il processo di compilazione del firmware si affida alla toolchain `arm-none-eabi-gcc`, integrata con il framework **Raspberry Pi RP2040 Arduino Core**. Quest'ultimo fornisce uno strato di compatibilità per le API Arduino sull'architettura RP2040, garantendo contestualmente l'accesso nativo alle librerie del *pico-sdk*, al bootloader USB e al file system *LittleFS*.

Al fine di automatizzare le procedure di compilazione e deployment, il repository include un *Makefile* principale. La logica di build è stata ingegnerizzata secondo criteri di modularità: l'automazione è parcellizzata in base all'ambito operativo e distribuita in moduli specifici con estensione `.mk`, allocati all'interno di sottodirectory dedicate.

Flag di Compilazione

Tre macro del preprocessore modificano il comportamento del firmware durante la compilazione:

Flag	Effetto
-DDEBUG=1	Abilita output diagnostico sulla seriale. Il task task_health è compilato solo con questo flag attivo.
-DDEFAULT_BUZZER_OFF	Imposta global_cfg.buzzer_enabled = false in config_init() e blocca peripherals_beep() con un return anticipato a compile-time, al fine di debuggare il codice e caricarlo senza disturbo.
-DFW_VERSION="\"...\"	Inserisce la stringa di versione nella view INFO. Generato automaticamente via git describe -tags -always.

Makefile: Target Principali

Uno schema semplificato del sistema di build e' rappresentato dai seguenti target:

```
# Compila e flasha con versione git automatica
flash:
pio -e pico --target upload

# Scarica meteo dall'API e lo sincronizza sul dispositivo
sync-meteo:
@python3 tools/api-weather/pcb_bridge.py

monitor:
pio device monitor
```

Listing 1 Makefile — target di sviluppo

Il make target sync-meteo separa nettamente acquisizione dati (script Python lato PC per l'API OpenMeteo) e visualizzazione (lato firmware): la board non contiene alcuna logica di rete, riceve tutto via porta seriale nel formato WXC, City, Temp, Wind, Hum, Code e risponde OK_WXC a conferma.

Coerentemente con gli obiettivi di prototipazione del sistema, le funzionalità Wi-Fi del modulo **ESP-01** integrato a bordo non sono state attivate direttamente nel firmware dell'unità principale. L'interazione con la rete è stata invece emulata tramite uno script Python operante come bridge seriale, il quale simula la comunicazione con un host remoto connesso a Internet. Grazie alla modularità del software, l'architettura risulta intrinsecamente predisposta per sviluppi futuri: l'integrazione effettiva del modulo ESP-01 richiederebbe unicamente la riconfigurazione del livello di trasporto (sostituendo il canale seriale cablato con un socket wireless), lasciando inalterata la logica applicativa di parsing e visualizzazione dei dati.

Versioning Firmware

La macro FW_VERSION è utilizzata in view_info.cpp con il fallback "v0.0.0-unknown" per build non provenienti da CI:

```
1 #ifndef FW_VERSION
2 #define FW_VERSION "v0.0.0-unknown"
3 #endif
4 canvas→print(F("VERSION: "));
5 canvas→print(FW_VERSION); // stringa in flash tramite macro F()
```

Listing 2 view_info.cpp — versione iniettata a compile-time

L'uso di `F()` (progmem wrapper di Arduino) mantiene la stringa "VERSIONE: " in flash invece di copiarla in SRAM.

3.3 PlatformIO e l'Ecosistema RP2040

Perché PlatformIO?

La configurazione dell'ecosistema è stata intenzionalmente centralizzata all'interno del file `platformio.ini`, escludendo qualsiasi dipendenza dalle impostazioni locali dell'IDE (VS Code). Questo approccio garantisce la totale portabilità del progetto e la riproducibilità del build system su qualsiasi macchina o pipeline di Continuous Integration (CI), indipendentemente dall'editor utilizzato dallo sviluppatore.

La sua funzione principale è eliminare la gestione manuale della toolchain; provvede autonomamente a scaricare compilatore, librerie, script di link e tool di upload nelle versioni esatte specificate.

I Due Framework per RP2040: la Scelta Pratica

Nel contesto di questo progetto il Raspberry Pi Pico è supportato da due framework basati su Arduino con filosofie molto diverse. PlatformIO consente di selezionare quello desiderato con una singola riga di configurazione, senza toccare il codice sorgente.

Mbed OS è il framework "ufficiale" di Arduino per Raspberry Pi Pico. Usa il `pico-sdk` sottostante ma lo avvolge nell'astrazione Mbed, che introduce un RTOS leggero⁶, un HAL proprio⁷ e convenzioni diverse. Il problema pratico è che il supporto dual-core tramite `setup1()/loop1()`, LittleFS nativa e le istanze `Serial1/Serial2` mappate liberamente su GPIO⁸ sono assenti o limitate.

Raspberry Pi RP2040 Arduino Core (di earlephilhower) è un porting alternativo, mantenuto dalla community, che espone direttamente le API `pico-sdk` sotto interfaccia Arduino senza lo strato Mbed. Supporta il dual-core tramite `setup1()/loop1()`, LittleFS con partizionamento configurabile, seriale su qualsiasi coppia di GPIO UART, e dà accesso diretto ai registri hardware nativi⁹ (`watchdog_hw`, `spin lock`, `PIO`) senza conflitti.

La configurazione in `platformio.ini` è la seguente:

```
[env:pico]
platform = https://github.com/maxgerhardt/platform-raspberrypi.git
board    = pico
framework = arduino
board_build.core = earlephilhower
```

Listing 3 platformio.ini — selezione framework e core

⁶Real-Time Operating System: un sistema operativo specializzato per applicazioni embedded in cui il tempo di esecuzione dei task e la gestione delle priorità sono deterministici, garantendo risposte in tempo reale agli eventi hardware.

⁷Hardware Abstraction Layer: uno strato software che interpone un livello di astrazione tra l'hardware fisico del silicio e l'applicazione, uniformando le API ma introducendo potenzialmente un overhead computazionale.

⁸General Purpose Input/Output: i pin digitali programmabili del microcontrollore che, nel caso dell'RP2040, possono essere associati dinamicamente a diverse periferiche interne (UART, I2C, SPI) tramite un multiplexer hardware interno.

⁹Strutture di memoria fisse collegate direttamente alla logica digitale del chip; la manipolazione diretta dei registri (*bare-metal*) scavalca le funzioni di astrazione del framework, azzerando le latenze e sbloccando le funzionalità specifiche del silicio, come le macchine a stati del PIO o i meccanismi di sincronizzazione hardware (`spin lock`).

Partizionamento della Flash

Firmware e filesystem non possono sovrapporsi, perciò il partizionamento è dichiarato nel `platformio.ini` e il linker script generato dal framework "Raspberry Pi RP2040 Arduino Core" segue quella definizione:

```
board_build.filesystem      = littlefs
board_build.filesystem_size = 0.5m      ; 512 KB per LittleFS
board_build.maximum_size    = 2097152   ; 2 MB totale
```

Listing 4 platformio.ini — layout flash

Con questi valori i primi 1.5 MB sono dedicati al firmware (codice + costanti), gli ultimi 512 KB sono la partizione LittleFS. In pratica, `LittleFS.begin()` monta la partizione a partire dall'offset calcolato e qualsiasi `open()/write()` rimane confinato in quello spazio.

Gestione delle Librerie

Le librerie esterne sono dichiarate nella sezione `lib_deps` di `platformio.ini`:

```
lib_deps =
adafruit/Adafruit BusIO @ ^1.17.4
adafruit/Adafruit GFX Library @ ^1.12.5
adafruit/Adafruit SSD1306 @ ^2.5.16
adafruit/Adafruit NeoPixel @ ^1.15.4
fastled/FastLED @ 3.9.4
olikraus/U8g2
```

Listing 5 platformio.ini — dipendenze librerie

PlatformIO scarica ciascuna libreria nella cartella `.firm-build/libdeps/` al primo build e le riutilizza nelle build successive.

La notazione `^1.17.4` significa "≥1.17.4 ma <2.0.0"; FastLED invece è fissato a 3.9.4.

La libreria `minmea` non appare in `lib_deps` perché è una dipendenza non presente nella raccolta di `platformio`; nel repository risiede direttamente in `lib/minmea/` e viene inclusa tramite `build_src_filter` e il flag `-I lib/minmea`.

Selezione dei File Sorgente: PlatformIO di default compila tutto ciò che trova in `src/`, tuttavia usare `build_src_filter` rende esplicito quali file compilare:

```
build_src_filter =
+<src/main.cpp>
+<src/core/*.cpp>
+<src/drivers/*.cpp>
+<src/ui/*.cpp>
+<src/ui/views/*.cpp>
+<src/pipeline/*.cpp>
+<lib/minmea/minmea.c>
```

Listing 6 platformio.ini — filtro sorgenti

Upload con picotool

Il metodo standard di upload del firmware è il drag-and-drop del file UF2 sulla USB Mass Storage che appare tenendo premuto BOOTSEL al reset. **picotool** è un tool ufficiale di Raspberry Pi che bypassa questo meccanismo e carica il firmware direttamente via USB senza richiedere BOOTSEL. In pratica:

1. Si esegue `pio run -target upload` (o `make flash`);

2. PlatformIO invoca `picotool load firmware.uf2`;
3. `picotool` parla con la board via USB, carica il firmware e forza il reboot;

```
upload_protocol = picotool
```

Listing 7 platformio.ini — upload tool

`picotool` può anche leggere la flash (`picotool read`), verificare il binario caricato.

3.4 Conflitti tra Arduino, pico-sdk e Librerie Esterne e relative soluzioni

La coesistenza di `arduino-pico`, `pico-sdk` e librerie come `FastLED` genera una serie di conflitti che il file `platformio.ini` risolve esplicitamente tramite `define` di preprocessore.

Assenza della funzione `timegm` in ambiente bare-metal

```
-D timegm=mktime
```

La funzione `timegm()` converte una struttura `struct tm` in un timestamp Unix assumendo che l'input sia espresso in UTC. Pur essendo comunemente disponibile su sistemi Linux e macOS, si tratta di un'estensione non inclusa nello standard ISO C.

Poiché la libreria `minmea` fa uso di `timegm()` all'interno di `minmea_gettime()`, è necessario fornire un'alternativa in fase di compilazione. La soluzione adottata consiste nel rimappare il simbolo su `mktime()` tramite una direttiva al preprocessore.

Nonostante dal punto di vista semantico le due funzioni differiscano - `mktime` interpreta l'input come ora locale e non come UTC - sull'RP2040 non è configurato alcun fuso orario. Di conseguenza, nel nostro specifico contesto operativo, il comportamento di `mktime()` risulta del tutto equivalente.

Buffer seriali e NMEA

```
-D SERIAL_RX_BUFFER_SIZE=256  
-D SERIAL_TX_BUFFER_SIZE=256
```

Il buffer UART di default in `arduino-pico` è 64 byte. Una sentence NMEA può essere di massimo 82 caratteri; con il buffer a 64 byte, una sentence "lenta" a processare sovrascrive il buffer prima che venga letta, producendo dati troncati o corrotti. Espandere a 256 byte garantisce che almeno tre sentence complete possano stare in coda senza perdite (a patto che il parsing sia altrettanto veloce).

Stack USB: TinyUSB

```
board_build.usb_stack = tinyusb
```

`arduino-pico` supporta due stack USB: quello interno del `pico-sdk` e `TinyUSB`. `TinyUSB` è preferibile, perché supporta CDC (porta seriale virtuale), MSC (mass storage per upload firmware) e HID simultaneamente.

3.5 Ottimizzazioni di co-design software-hardware

Librerie e Dipendenze Software

Per l'implementazione del middleware e dei driver di periferica, sono state selezionate librerie ottimizzate per l'ambiente *bare-metal*, privilegiando l'efficienza nell'uso delle risorse e la stabilità del sistema a lungo termine.

Parsing GNSS: minmea Per l'estrazione delle coordinate geografiche e del timestamp UTC dalle frasi NMEA fornite dal modulo SAM-M8Q [1], è stata integrata la libreria minmea [9]. Si tratta di un parser scritto in C99, estremamente leggero e adatto a sistemi *resource-constrained* poiché non fa uso di memoria dinamica (*heap*), evitando così problemi di frammentazione della RAM durante sessioni di logging prolungate.

Per elaborare i dati in arrivo dalla UART, il firmware identifica l'intestazione della frase e la passa al parser per la conversione in strutture dati fruibili.

L'utilizzo di minmea garantisce che il parsing avvenga in un intervallo temporale deterministico, riducendo la latenza di elaborazione e assicurando che il Core 1 possa gestire simultaneamente gli interrupt della seriale e la logica di controllo senza perdita di pacchetti.

Interfaccia Visuale: SSD1306 e Adafruit GFX La comunicazione visiva con l'utente è affidata a un display OLED interfacciato tramite il bus seriale I2C0 nativo del SoC RP2040. Al fine di astrarre la complessità legata alla renderizzazione dei font alfanumerici e delle primitive grafiche, il firmware integra le librerie Adafruit GFX [10] e Adafruit SSD1306 [11].

In fase di boot, la periferica viene inizializzata impostando l'indirizzo slave I2C (0x3C) e allocando nello spazio di memoria RAM del microcontrollore il buffer necessario alla gestione dei pixel.

Per ottimizzare le prestazioni complessive del sistema e mitigare i vincoli di banda del canale seriale, il firmware adotta un'architettura basata su *framebuffer* locale. Tutte le operazioni di scrittura e disegno geometrico modificano esclusivamente la matrice di pixel allocata nella RAM dell'RP2040. Il trasferimento fisico dei dati verso la GDDRAM (*Graphic Display Data RAM*) del controller SSD1306 avviene in modo atomico e sequenziale solo a seguito dell'invocazione della funzione di gestione `display_show()` (la quale agisce da wrapper ed esegue internamente il metodo `canvas->display()` sul puntatore della libreria Adafruit). Questo approccio centralizzato riduce al minimo l'overhead di comunicazione sul bus I2C — rendendo il trasferimento dei dati un'operazione concentrata nel tempo — e previene fenomeni di sfarfallio (*flickering*) visivo durante l'aggiornamento in tempo reale delle coordinate GNSS e dei parametri ambientali.

Feedback Cromatico: FastLED e Offloading PIO Per la gestione dei LED WS2812B è stata scelta la libreria FastLED [12], che permette una manipolazione avanzata dello spazio colore HSV (più intuitivo dell'RGB per creare gradienti). La generazione fisica dei segnali di timing è delegata alle macchine a stati del PIO, mentre la libreria fornisce l'astrazione per definire i colori.

Il sistema utilizza i LED per fornire un feedback immediato sull'integrità del sistema: un gradiente dal rosso al verde indica la precisione del fix satellitare, mentre brevi impulsi luminosi segnalano l'avvenuto salvataggio di un log sulla memoria Flash, garantendo all'utente il monitoraggio dello stato del dispositivo senza dover consultare il display OLED.

Filesystem: LittleFS Per la gestione dell'archiviazione dei log e di parametri da salvare in memoria non volatile (Flash W25Q16JV [13]), si rende necessario l'uso di un filesystem. È stato scelto il filesystem LittleFS [14], ottimizzato e pensato per memorie con bassa vita in scrittura (100.000 scritture per settore nel nostro caso); esso fa un uso minimo della ROM ed è indipendente dalle dimensioni del FS, dimostrandosi resiliente a perdite di alimentazione improvvise. La documentazione per l'uso dell'API si trova qui [15].

Il programma utilizza la memoria flash per salvare l'ultima posizione valida del GPS in modo che ne sia sempre una di fallback. Anche i dati meteo ritornati dal programma sul PC via UART vengono salvati in flash, permettendo di mantenerli anche se il PC viene scollegato e la board spenta.

Offloading Computazionale tramite PIO

Una delle sfide critiche nello sviluppo del firmware è stata la gestione della concorrenza tra il protocollo dei LED **WS2812B** e il flusso dati asincrono del modulo GNSS. I LED richiedono una modulazione degli impulsi con timing estremamente rigorosi (nell'ordine dei nanosecondi). Le implementazioni software tradizionali (*bit-banging*) soddisfano questi requisiti entrando in una sezione critica in cui vengono **disabilitati globalmente gli interrupt**.

Tale approccio è incompatibile con il data-logging GNSS: la disabilitazione degli interrupt, anche per pochi millisecondi, causa l'overflow del buffer della UART, portando alla perdita di byte fondamentali nelle frasi NMEA. Per ovviare a questo limite, è stato utilizzato il sottosistema **PIO** (*Programmable I/O*) dell'RP2040:

- **Parallelismo hardware e Determinismo:** Una macchina a stati dedicata (*State Machine*) esegue un programma PIO in assembly dedicato per generare le forme d'onda. Questo processo avviene in modo totalmente indipendente dai cicli di clock dei core ARM, garantendo un segnale privo di *jitter*.
- **Comunicazione tramite FIFO:** Il *Core 0* carica i dati cromatici in un buffer hardware di tipo **FIFO** (*First-In-First-Out*). Una volta trasferito il dato, la CPU torna immediatamente a gestire il parsing NMEA, mentre il PIO svuota il buffer pilotando fisicamente il pin.

Sincronizzazione e Protocollo WS2812B L'implementazione sfrutta un programma PIO che traduce ogni bit di colore in una sequenza di impulsi HIGH/LOW con rapporto ciclico (*duty cycle*) variabile. Questo permette di delegare interamente la gestione del tempo all'hardware dedicato.

Questa architettura realizza un vero e proprio **offloading hardware**: il processore non deve più attendere il completamento della trasmissione seriale verso i LED (operazione intrinsecamente lenta e bloccante), ma può dedicarsi esclusivamente ad altre operazioni.

Gestione dei Conflitti di Interrupt In un sistema embedded senza PIO, l'arrivo di un byte sulla UART durante la trasmissione verso i LED genererebbe una **Interrupt Latency** inaccettabile. Se l'interrupt venisse servito, il segnale dei LED si corromperebbe (causando sfarfallio); se venisse ignorato, il byte UART andrebbe perso.

L'offloading su PIO risolve il dilemma alla radice: poiché il PIO non può essere interrotto da eventi software, il protocollo fisico dei LED rimane stabile, mentre i core ARM restano sempre disponibili per rispondere istantaneamente agli interrupt della UART e dei pulsanti GPIO (*falling edge*), garantendo un sistema realmente *real-time*.

SystemDataPacket

La struttura dati principale dedicata alla telemetria è definita nel file `messages.h`. A questa definizione viene applicato l'attributo `__attribute__((packed))` per forzare un layout compatto in memoria:

```
1  typedef struct __attribute__((packed)) {
2      double    latitude;
3      double    longitude;
4      float     altitude_m;
5      float     speed_ms;
6      uint8_t   satellites;
7      uint8_t   gps_status;
8      float     pitch, roll, yaw;
9      float     battery_v;
10     float     temp_c;
```

```
11     uint32_t uptime_s;  
12     WeatherDataPacket weather;  
13     error_report_t last_error;  
14     struct {  
15         uint8_t is_armed      : 1;  
16         uint8_t is_logging   : 1;  
17         uint8_t wifi_active  : 1;  
18         uint8_t error_active : 1;  
19     } flags;  
20 } SystemDataPacket;
```

Listing 8 messages.h — Struttura telemetrica senza padding

L'utilizzo del tipo `double` (a 64 bit) per le coordinate geografiche rappresenta una scelta progettuale ben precisa. Il tipo `float` standard garantisce infatti circa 7 cifre significative totali, un valore a bassa precisione per le coordinate globali.

Ad esempio, in una latitudine europea come 46.073452° , la parte intera impegna già le prime due cifre significative, lasciandone solo cinque per la parte decimale (il che si traduce in una risoluzione spaziale limitata a circa 1.1 metri).

Il formato `double`, offrendo 15 cifre significative, porta la risoluzione teorica a un livello millimetrico. Sebbene questo grado di dettaglio superi l'accuratezza del modulo GPS SAM-M8Q, risulta fondamentale per prevenire la perdita di precisione e garantire un *logging* accurato.

Per ottimizzare lo spazio occupato, il campo `flags` sfrutta la sintassi dei *bit-field*, raggruppando quattro indicatori di stato booleani all'interno di un singolo byte e mantenendo un'elevata leggibilità del codice.

Infine, l'impiego della direttiva `packed` impone al compilatore di eliminare qualsiasi byte di riempimento (*padding*) tra i campi contigui. Ciò si traduce in un *layout* di memoria compatto e strettamente deterministico, requisito essenziale per preservare l'integrità dei dati durante la serializzazione binaria verso la memoria Flash e nelle comunicazioni IPC.

3.6 Periferiche e Persistenza in memoria

Per la memorizzazione non volatile dei log GNSS e dei parametri di sistema, il progetto integra una memoria Flash NOR **Winbond W25Q16JV** [13]. Il chip, con una capacità di 16 Mbit (2 MB), comunica con l'RP2040 tramite interfaccia *Quad-SPI* (QSPI), garantendo un throughput di dati elevato e latenze ridotte durante le operazioni di scrittura intensiva.

Modulo GNSS: u-blox SAM-M8Q

Il sistema di posizionamento si affida al modulo **u-blox SAM-M8Q** [1], un ricevitore GNSS (*Global Navigation Satellite System*) di fascia professionale che integra un'antenna *patch* ceramica pre-accordata.

L'integrazione di questo componente offre diversi vantaggi tecnologici fondamentali per il progetto:

- **Ricezione Multi-Costellazione:** Il modulo supporta la ricezione simultanea di tre sistemi GNSS (GPS, Galileo e GLONASS). Questa capacità permette di agganciare un numero superiore di satelliti rispetto ai ricevitori a singola costellazione, garantendo una precisione sub-metrica e un fix della posizione estremamente rapido (*Time To First Fix*) anche in condizioni di scarsa visibilità del cielo.
- **Protocollo di Comunicazione:** Il modulo comunica con il microcontrollore tramite interfaccia UART. Le sentenze **NMEA 0183** vengono trasmesse serialmente e processate dal firmware per estrarre coordinate, altitudine e timestamp UTC.
- **Integrità del Segnale:** Il modulo è alimentato tramite la linea a 3,3 V filtrata dall'LDO dedicato. La scelta di un'antenna integrata minimizza le perdite di segnale dovute a disadattamenti di impedenza, tipiche delle soluzioni con antenna esterna e connettore SMA.

L'output seriale del SAM-M8Q viene gestito in modo asincrono dal *Core 1* dell'RP2040 [2], permettendo di mantenere un flusso di log costante sulla Flash senza interferire con le altre funzioni del sistema.

File System: Integrità e Wear Leveling con LittleFS

Data la natura della memoria Flash, soggetta a un numero finito di cicli di cancellazione (10^5 cicli per settore), la gestione dei dati non avviene tramite scrittura grezza, ma attraverso il filesystem **LittleFS**. L'adozione di questo filesystem *open-source*, ottimizzato per microcontrollori, offre vantaggi cruciali per un data-logger professionale:

- **Wear Leveling:** LittleFS distribuisce uniformemente le operazioni di scrittura su tutti i blocchi della memoria, evitando l'usura precoce dei settori iniziali e prolungando drasticamente la vita utile della W25Q16JV.
- **Power-loss Resilience:** Il file system è progettato per gestire interruzioni improvvise dell'alimentazione. Grazie a una struttura "copy-on-write", i dati preesistenti non vengono sovrascritti finché la nuova operazione non è completata, garantendo l'integrità dei log anche in caso di distacco della batteria.
- **Gestione dei Bad Blocks:** Il sistema è in grado di rilevare e isolare dinamicamente eventuali blocchi danneggiati della Flash, mantenendo l'affidabilità del logging nel tempo.

Come altra misura preventiva, la scrittura su `/gps.log` passa attraverso due filtri distinti prima di toccare la flash.

Il primo, in `storage_log_gps_fix()`, scarta coordinate troppo vicine a (0,0) con una tolleranza di 10^{-4} gradi. Questo esclude la risposta di default del SAM-M8Q quando non ha segnale, che trasmette comunque sentence GGA con latitudine e longitudine a zero.

Il secondo, in `check_and_log_gps()`, confronta la nuova posizione con l'ultima salvata e scrive solo se la variazione supera 0.005° in latitudine o longitudine:

```
1 static void check_and_log_gps(float lat, float lon) {  
2     static float last_lat = 0, last_lon = 0;  
3     if (abs(lat - last_lat) > 0.005f || abs(lon - last_lon) > 0.005f) {  
4         storage_log_gps_fix(lat, lon);  
5         last_lat = lat; last_lon = lon;  
6     }  
7 }
```

Listing 9 telemetry.cpp — filtro sul log GPS

3.7 Sistema di Telemetria GPS

`gps_hw_enable()` gestisce la sequenza di accensione del SAM-M8Q: attiva `SAM_EN_PIN` con logica negata (PMOS: LOW = ON), poi esegue un reset fisico via `SAM_RST_PIN` (LOW per 100 ms, poi HIGH) seguito da 1 s di attesa per la stabilizzazione dell'oscillatore interno. `Serial2` è configurato su GPIO 4/5 a 9600 baud, il rate standard per il SAM-M8Q.

Rate Limiting e Parsing NMEA

`telemetry_update()` legge al massimo 64 byte da `Serial2` per iterazione. Senza questo limite, un burst di sentence NMEA potrebbe saturare il loop di *Core 1* per decine di millisecondi, ritardando l'aggiornamento del dato condiviso fra i cores. I byte sono accumulati in `nmea_buffer[128]` fino al carattere `'\n'`, poi `process_nmea()` analizza la sentence.

Utilizziamo la stringa GGA perché fornisce il campo `satellites_tracked`, altrimenti assente in RMC. Il parser considera valida esclusivamente una frase **GGA** con `fix_quality > 0`.


```

1  static void process_nmea(const char* line) {
2      struct minmea_sentence_gga gga;
3      if (minmea_parse_gga(&gga, line) && gga.fix_quality > 0) {
4          float lat = minmea_tocoord(&gga.latitude);
5          float lon = minmea_tocoord(&gga.longitude);
6          uint32_t save = spin_lock_blocking(telemetry_lock);
7          telemetry_data.latitude = lat;
8          telemetry_data.longitude = lon;
9          telemetry_data.gps_status = true;
10         telemetry_data.satellites = gga.satellites_tracked;
11         last_fix_timestamp = millis();
12         spin_unlock(telemetry_lock, save);
13         check_and_log_gps(lat, lon);
14     }
15 }

```

Listing 10 telemetry.cpp — validazione fix GGA

Due soglie temporali indipendenti gestiscono la perdita di segnale:

```

1  if (telemetry_data.gps_status &&
2      (millis() - last_fix_timestamp > 3000)) {
3      telemetry_data.gps_status = false;
4      telemetry_data.satellites = 0;
5  }
6
7  bool telemetry_is_healthy() {
8      if (last_fix_timestamp == 0) return true; // cold start
9      return (millis() - last_fix_timestamp) < 5000;
10 }

```

Listing 11 telemetry.cpp — timeout fix e health check

Il caso `last_fix_timestamp == 0` (nessun fix mai ricevuto) è trattato come stato sano: il sistema è in cold start e segnalare un errore prima che il modulo abbia avuto tempo di acquisire i satelliti produrrebbe un falso errore.

3.8 Architettura Multi-Core: organizzazione software

Divisione dei Ruoli

La separazione dei ruoli fra cores è interamente software. Il file `main.cpp` mappa le funzioni standard di Arduino (`setup` e `loop`) tramite le seguenti definizioni appartenenti al framework (earlphilhower):

```

1  void setup1() { core1_setup(); }
2  void loop1() { core1_loop(); }
3
4  void setup() {
5      watchdog_hw->ctrl &= ~(1 << 30); // disabilita WDT
6      core0_setup();
7  }
8  void loop() { core0_loop(); }

```

Listing 12 main.cpp — mapping dual-core

Core 0 (master) gestisce lo scheduler dei tasks, la UI, la comunicazione seriale, l'uptime, i LED e LittleFS. Esegue anche `telemetry_update()` in `core0_loop()` come parte dell'update rapido dei sensori locali.

Core 1 (worker) esegue un loop semplice e bloccante: chiama `telemetry_update()` per leggere la UART GPS e processare le frasi NMEA; dopodiché trasferisce il risultato al Core 0 via IPC alla fine di ogni iterazione.

IPC: Spin Lock su Memoria Condivisa

Come visto nella sezione (2.2), l'RP2040 offre due meccanismi IPC hardware:

- **FIFO inter-core**
- **Spin locks**

La scelta ricade sugli spin lock con memoria condivisa per una ragione pratica: `SystemDataPacket` è una struct di oltre 80 byte. Trasferirla via FIFO implicherebbe una serializzazione in parole da 32 bit e riassettaggio con corrispondente overhead e possibili race condition.

Attraverso la memoria condivisa, invece, il trasferimento non è più necessario:

```

1  static SystemDataPacket shared_data;
2  static spin_lock_t* data_lock =
3  spin_lock_init(spin_lock_claim_unused(true));
4
5  bool sys_manager_send_data(SystemDataPacket* packet) {
6      uint32_t save = spin_lock_blocking(data_lock);
7      shared_data = *packet;
8      spin_unlock(data_lock, save);
9      return true;
10 }
```

Listing 13 `system_manager.cpp` — accesso bloccante alla struct condivisa

Doppio Spin Lock e separazione degli ambiti: Sono presenti due spin lock distinti: `telemetry_lock` in `telemetry.cpp` protegge il buffer interno di Core 1 e `data_lock` in `system_manager.cpp` protegge la zona condivisa. Core 1 aggiorna `telemetry_data` usando `telemetry_lock` durante il parsing NMEA, e trasferisce verso `shared_data` sotto `data_lock` solo quando disponibile un dato verificato.

I due lock non sono mai acquisiti contemporaneamente, escludendo il rischio di deadlock.

Strategia di Logging e Concorrenza fra Cores

Nel firmware sviluppato, la scrittura su LittleFS è affidata esclusivamente al Core 0 per garantire la massima stabilità del filesystem. Per evitare che le latenze intrinseche della memoria Flash (cancellazione settori) degradino le prestazioni del parsing NMEA, è stata implementata una netta separazione dei compiti: il Core 1 si occupa del parsing asincrono dei dati GNSS, mentre il Core 0 funge da orchestratore per le operazioni di I/O.

Il log dei dati avviene tramite un approccio *event-driven*: la scrittura su file (`/gps.log`) è condizionata da una logica di *dead-band* (0.005° di tolleranza spaziale), che minimizza drasticamente il numero di cicli di scrittura. Questa scelta evita la necessità di complessi buffer circolari in RAM, mantenendo il firmware snello e resiliente. Il trasferimento dei dati validati dal Core 1 al Core 0 avviene tramite la memoria condivisa protetta dai `data_lock`, assicurando che l'integrità del log sia preservata senza bloccare il flusso di acquisizione del segnale satellitare.

3.9 Scheduler dei Task e Cooperazione Multitasking

Design dell'Architettura Cooperativa

Per garantire un'esecuzione deterministica delle routine periodiche senza introdurre l'overhead computazionale e la complessità di memorizzazione di un RTOS preemptivo completo, il sistema adotta uno scheduler cooperativo minimale basato su time-slicing. Lo scheduler è implementato con logica *header-only* nel file `scheduler.h` mediante funzioni dichiarate `inline`, ottimizzando l'allineamento in memoria ed eliminando il costo di chiamata a funzione (*function call overhead*) durante il ciclo di esecuzione principale.

La struttura dati fondamentale e il motore di verifica del tempo sono definiti all'interno della struct `Task`:

```

1  typedef void (*TaskCallback)();
2
3  typedef struct {
4      TaskCallback callback; // Puntatore alla funzione associata al task
5      uint32_t interval;     // Periodo di attivazione espresso in millisecondi
6      uint32_t last_run;     // Timestamp (millis) dell'ultima esecuzione completata
7  } Task;
8
9  inline void run_task(Task* t) {
10     uint32_t now = millis();
11     // Gestione nativa del rollover del timer hardware tramite aritmetica dei moduli
12     if (now - t->last_run ≥ t->interval) {
13         t->callback();
14         t->last_run = now;
15     }
16 }
```

Listing 14 `scheduler.h` — Struttura dati e motore di esecuzione dei task

L'istanza di ciascun task sfrutta le espressioni lambda introdotte dallo standard C++11, permettendo di incapsulare in modo conciso la logica di interfacciamento direttamente in fase di allocazione statica:

```

1  static Task task_ui = {[]() {
2      SystemDataPacket frame;
3      if (sys_manager_receive_data(&frame)) {
4          ui_manager_update(&frame);
5      }
6  }, 33, 0}; // Frequenza di aggiornamento mirata a ~30 Hz
```

Listing 15 `core0_manager.cpp` — Allocazione statica del task grafico dell'interfaccia utente

Pianificazione dei Task sul Core 0

Il ciclo di esecuzione principale del *Core 0* (`core0_loop`) interroga sequenzialmente le funzioni di controllo dei singoli task registrati. Nella Tabella 1 viene riassunto il piano di schedulazione del core primario, evidenziando i periodi di attivazione e le responsabilità sistemiche associate.

Analisi critica del `task_uptime`: L'analisi del firmware evidenzia un'inefficienza architetturale nella temporizzazione del modulo `task_uptime`. Sebbene il task venga invocato dallo scheduler con un periodo di 100 ms, la funzione di callback associata implementa una guardia logica interna supplementare (`millis() - last_uptime_tick ≥ 1000`), la quale di fatto ne vincola l'esecuzione utile a intervalli di un secondo.

Questo pattern genera un rapporto di utilità del 10 %, introducendo chiamate ridondanti che impegnano la CPU senza eseguire calcoli effettivi. Al fine di ottimizzare il consumo computazionale e semplificare il flusso logico, la soluzione ottimale prevede l'innalzamento del periodo del task direttamente a 1000 ms all'interno della struttura dati, eliminando completamente il controllo condizionale annidato nella callback.

Tabella 1 Task registrati ed eseguiti nel Core 0.

Task Identificativo	Periodo	Responsabilità Funzionale
task_serial	10 ms	Parsing e processing dei comandi asincroni da terminale seriale (PC host).
task_input	20 ms	Polling dell'interfaccia di input e smistamento dei segnali di debouncing alla UI.
task_ui	33 ms	Rendering grafico dei widget dell'interfaccia utente su display OLED (≈ 30 Hz).
task_uptime	100 ms	Incremento del contatore di attività globale (sub-ottimizzato, vedi analisi).
task_leds	100 ms	Gestione dei feedback visivi RGB e processing dinamico degli effetti di stato.
task_health	10 000 ms	Diagnostica di sistema (Heartbeat) attiva solo in modalità DEBUG.

3.10 Sistema di Gestione e Codifica degli Errori

Per garantire il monitoraggio dello stato di salute del sistema (*health monitoring*) e facilitare le operazioni di diagnostica inter-core sul bus IPC, il firmware implementa una codifica strutturata delle anomalie nel file `error_codes.h`. Il design si basa su un approccio gerarchico che mappa i codici di errore in base alla macro-categoria di appartenenza, come sintetizzato nella Tabella 2.

Tabella 2 Classificazione gerarchica dei codici di errore.

Spazio dei Codici	Macro-Categoria	Esempi Notevoli
0x10–0x1F	Sistema (Core/IPC)	ERR_SYS_IPC_TIMEOUT, ERR_SYS_WATCHDOG
0x20–0x2F	Hardware (Driver)	ERR_HW_DISPLAY_LOST, ERR_HW_I2C_BUS_LOCK
0x30–0x3F	Sensori (Acquisizione)	ERR_SENS_GPS_NO_DATA, ERR_SENS_GPS_BAD_FIX
0x40–0x4F	Power (Alimentazione)	ERR_PWR_BATT_LOW, ERR_PWR_BATT_CRITICAL
0x50–0x5F	Storage (File System)	ERR_STG_FS_CORRUPT, ERR_STG_WRITE_FAIL

Questa architettura firmware consente di identificare l'origine dell'anomalia analizzando esclusivamente il *nibble* superiore (i 4 bit più significativi) del codice d'errore, eliminando la necessità di computare onerose tabelle di decodifica in esecuzione. L'estrazione della macro-categoria avviene a runtime tramite un'operazione bitwise di scorrimento a destra:

$$\text{categoria} = \text{error_code} \gg 4 \quad (1)$$

Tale informazione può essere così renderizzata immediatamente sul display OLED o instradata sul flusso di telemetria seriale.

Di seguito si riporta l'implementazione formale delle strutture dati e dei registri enumerativi sviluppati nel file di intestazione:

```

1  #ifndef ERROR_CODES_H
2  #define ERROR_CODES_H
3
4  #include <stdint.h>
5  #include <stdbool.h>
6
7  /**
8   * @brief Categorie principali di errore mappate sui rispettivi nibble alti.
9   */
10 typedef enum {

```

```

11     ERR_CAT_NONE      = 0x00, /**< Nessun errore rilevato. */
12     ERR_CAT_SYS       = 0x01, /**< Errori critici di sistema (Kernel/IPC). */
13     ERR_CAT_HW        = 0x02, /**< Malfunzionamenti driver o periferiche fisiche. */
14     ERR_CAT_SENSORS   = 0x03, /**< Anomalie acquisizione dati (GPS/IMU). */
15     ERR_CAT_POWER     = 0x04, /**< Anomalie elettriche o condizioni di batteria. */
16     ERR_CAT_STORAGE   = 0x05 /**< Errori del file system LittleFS o supporti esterni. */
17 } error_category_t;
18
19 /**
20  * @brief Codici di errore specifici suddivisi per domini numerici.
21  */
22 typedef enum {
23     SUCCESS = 0,
24
25     // --- Sistema (0x10-0x1F) ---
26     ERR_SYS_IPC_TIMEOUT = 0x10, /**< Timeout di sincronizzazione inter-core. */
27     ERR_SYS_WATCHDOG    = 0x11, /**< Intervento del Watchdog hardware. */
28     ERR_SYS_OUT_OF_MEMORY = 0x12, /**< Fallimento nell'allocazione dinamica della RAM.
29                                     */
30
31     // --- Hardware (0x20-0x2F) ---
32     ERR_HW_I2C_BUS_LOCK = 0x20, /**< Bus I2C in stato di stallo (SDA/SCL low). */
33     ERR_HW_DISPLAY_LOST = 0x21, /**< Mancata risposta del controller SSD1306. */
34     ERR_HW_UART_OVERRUN = 0x22, /**< Buffer di ricezione UART saturo. */
35
36     // --- Sensori (0x30-0x3F) ---
37     ERR_SENS_GPS_NO_DATA = 0x30, /**< Assenza di stringhe NMEA sul canale GPS. */
38     ERR_SENS_IMU_NOT_CAL = 0x31, /**< Mancata calibrazione dei vettori inerziali. */
39     ERR_SENS_GPS_BAD_FIX = 0x32, /**< Degradazione della precisione geometrica (HDOP
40                                     elevato). */
41
42     // --- Power (0x40-0x4F) ---
43     ERR_PWR_BATT_LOW     = 0x40, /**< Tensione di batteria sotto la soglia nominale. */
44     ERR_PWR_BATT_CRITICAL = 0x41, /**< Soglia critica di scarica (Failsafe trigger). */
45     ERR_PWR_USB_OVERVOLT = 0x42, /**< Rilevamento sovratensione sulla linea VBUS. */
46
47     // --- Storage (0x50-0x5F) ---
48     ERR_STG_SD_NOT_FOUND = 0x50, /**< Supporto di memoria non rilevato all'avvio. */
49     ERR_STG_WRITE_FAIL   = 0x51, /**< Fallimento della transazione di scrittura su file
50                                     . */
51     ERR_STG_FS_CORRUPT   = 0x52 /**< Incoerenza logica o corruzione del file system.
52                                     */
53 } error_code_t;
54
55 /**
56  * @brief Struttura dati per il packaging e la trasmissione dei report diagnostici.
57  */
58 typedef struct {
59     error_category_t category; /**< Macro-categoria estratta dal codice. */
60     error_code_t code; /**< Identificativo univoco dell'errore. */
61     uint32_t timestamp; /**< Timestamp di occorrenza (millis)
62
63     bool is_critical; /**< Flag di severita per routine di Failsafe
64 } error_report_t;
65
66 #endif /* ERROR_CODES_H */

```

Listing 16 error_codes.h — Definizione dei tipi enumerativi e delle strutture per la diagnostica

4 Interfaccia Utente

4.1 FSM e Contratto `view_interface_t`

Il sistema UI è una FSM dove ogni stato corrisponde a una schermata. `view_id_t` enumera le view disponibili; il puntatore `current_view` in `ui_manager.cpp` punta all'istanza `view_interface_t` attiva. Le transizioni avvengono solo tramite `ui_manager.navigate_to()` che chiama la funzione `on_exit` della view uscente, aggiorna il puntatore, attiva il feedback sonoro di navigazione e infine esegue la funzione `on_enter` della nuova view.

```

1  typedef struct {
2      void (*on_enter)(void);
3      void (*on_update)(const void* data);
4      void (*on_input)(button_t btn, button_state_t state);
5      void (*on_exit)(void);
6  } view_interface_t;

```

Listing 17 `view_definitions.h` — contratto funzionale di una view

4.2 Rendering e Timing OLED

Il ciclo di render segue il pattern `clearDisplay()` → scrittura nel frame buffer → `display()`. La chiamata `display()` trasferisce 1024 byte (128×64 bit) via I2C a 400 kHz: circa 25 ms di trasferimento. Per questo `task_ui` è programmato ogni 33 ms, evitando di accodare un secondo trasferimento prima che il precedente sia completato.

4.3 View Implementate

view_home implementa un menu a cursore con navigazione ciclica. La riga selezionata è renderizzata in modalità inversa (rettangolo bianco riempito + testo nero), tecnica che non richiede bitmap. La mappatura tra posizione del cursore e destinazione è una lookup table:

```

1  view_id_t targets[] = {VIEW_ID_GPS, VIEW_ID_METEO, VIEW_ID_SETTINGS};
2  ui_manager.navigate_to(targets[menu_cursor]);

```

view_gps converte la velocità (`speed_ms * 3.6f`), formatta le coordinate con 6 decimali e mostra il conteggio satelliti con padding fisso a 2 cifre (`%02d`).

view_meteo legge il campo `weather.valid` come sentinella. La descrizione meteo è derivata con un sistema a soglie da `weather_code`: 0 = Clear Sky, 1–3 = Partly Cloudy, ≥60 = Rainy. Le soglie seguono la classificazione WMO usata da `OpenMeteo`.

view_settings gestisce 5 voci con ciclo della luminosità secondo la formula:

```

1  global_cfg.oled_brightness =
2      (global_cfg.oled_brightness ≥ 250)
3      ? 50
4      : ((global_cfg.oled_brightness / 50) + 1) * 50;

```

Il risultato è una sequenza circolare 50 → 100 → 150 → 200 → 250 → 50 con incrementi di 50. L'effetto è applicato immediatamente su hardware tramite `display_set_brightness(global_cfg.oled_brightness)`

view_info mostra versione firmware e uptime, formattato HH:MM:SS con divisioni intere su `uptime_s`.

Overlay Errori

`ui_manager_update()` controlla `data->flags.error_active` prima di delegare alla view normale. Se attivo e non confermato, `render_error_overlay()` sovrascrive il buffer mostrando categoria e codice in esadecimale. Al primo input successivo, `ui_manager_dispatch_input()` imposta `error_acknowledged` e chiama `sys_manager_clear_error()`, ripristinando la navigazione.

Feedback Acustico

I beep sono differenziati per tipo di interazione in `ui_manager_dispatch_input()`: 2400 Hz/15 ms per navigazione (UP/DOWN), 1800 Hz/20 ms per BACK. La navigazione verso una nuova view genera 1500 Hz/80 ms.

5 Interfacciamento PC-Board: API Python e Protocollo Seriale

5.1 Architettura del Sistema di Sincronizzazione

Il firmware non ha connettività di rete diretta. La sincronizzazione dei dati meteo e il recupero della posizione GPS avvengono tramite la porta seriale USB, che il Pico espone come CDC (Communication Device Class).

Il flusso è di tipo **master-slave**: il PC chiede, la board risponde o riceve; non c'è polling autonomo dalla board verso il PC.

PC		RP2040
pcb_bridge.py	→ GET_GPS\n	sys_manager_handle_serial()
	← GPS_DATA:LIVE,lat,lon	
Nominatim API		
OpenMeteo API	→ WXC,City,T,W,H,C\n	
	← OK_WXC	

Figura 1 Sequenza di scambio dati tra pcb_bridge.py e firmware.

Protocollo di comunicazione

Il protocollo creato è in puro ASCII, ed ogni messaggio è terminato da '\n'.

Direzione	Messaggio	Significato
PC → Board	GET_GPS	Richiede posizione corrente
Board → PC	GPS_DATA:LIVE,lat,lon	Fix GPS valido, coordinate correnti
Board → PC	GPS_DATA:CACHE,lat,lon	Nessun fix, coordinate da Flash
PC → Board	WXC,City,T,W,H,C	Pacchetto meteo (5 campi CSV)
Board → PC	OK_WXC	Conferma ricezione meteo
PC → Board	GET_STATUS	Richiesta stato diagnostico
Board → PC	(multiriga)	Dump errori e stato sistema

5.2 Lo Script Python

tools/api-weather/pcb_bridge.py è il programma lato PC che orchestra l'intera sequenza. Le uniche dipendenze esterne sono pyserial e requests per le API HTTP.

```

1  def main():
2      ser = serial.Serial("/dev/ttyACM0", 115200, timeout=5)
3      time.sleep(5)          # attende il boot del firmware post-reset USB
4      ser.reset_input_buffer()
5
6      # 1. Richiede la posizione GPS alla board
7      ser.write(b"GET_GPS\n")
8
9      data_line = None
10     for _ in range(50):      # attende fino a 50 righe in risposta

```

```

11 line = ser.readline().decode('utf-8', errors='ignore').strip()
12 if line.startswith("GPS_DATA:"):
13     data_line = line.replace("GPS_DATA:", "")
14     break
15
16 # data_line e' "LIVE,lat,lon" oppure "CACHE,lat,lon"
17 source, lat, lon = data_line.split(',')[0], *map(float, data_line.split(',')[1:])
18
19 # 2. coordinate → nome città
20 city = get_city_name(lat, lon) # Nominatim (OpenStreetMap)
21
22 # 3. Meteo dalla posizione
23 cur = fetch_weather(lat, lon) # OpenMeteo API
24
25 # 4. Formatta e invia alla board
26 wxc = f"WXC,{city},{cur['temperature_2m']},{cur['wind_speed_10m']},"
27 wxc += f"{cur['relative_humidity_2m']},{cur['weather_code']}\n"
28 ser.write(wxc.encode('utf-8'))

```

Listing 18 pcb_bridge.py — flusso principale

NOTA 1: `time.sleep(5)` dopo l'apertura della porta seriale è necessario poiché quando il PC apre la connessione CDC, il Pico riceve un segnale DTR che interpreta come richiesta di reset. Il firmware riparte da capo, esegue l'intera sequenza di boot (LittleFS, display, animazione) e solo dopo qualche secondo il sistema è in uno stato stabile pronto a processare comandi. Senza questo delay, GET_GPS verrebbe inviato durante il boot e ignorato o perso nel buffer.

NOTA 2: Il loop `for _ in range(50)` legge fino a 50 righe cercando quella che inizia con `GPS_DATA:`. Infatti le righe precedenti possono contenere output di debug del firmware che vengono silenziosamente scartate.

API HTTP esterne

```

1 def get_city_name(lat, lon):
2     # Nominatim: reverse geocoding con OpenStreetMap
3     url = "https://nominatim.openstreetmap.org/reverse"
4     params = {"lat": lat, "lon": lon, "format": "json", "zoom": 10}
5     headers = {"User-Agent": "PPSE-Lab-Weather-Bridge"}
6     r = requests.get(url, params=params, headers=headers, timeout=5)
7     address = r.json().get("address", {})
8     return address.get("city") or address.get("town") or address.get("village") or "
9         Unknown"
10
11 # OpenMeteo
12 def fetch_weather(lat, lon):
13     url = "https://api.open-meteo.com/v1/forecast"
14     params = {
15         "latitude": lat, "longitude": lon,
16         "current": "temperature_2m,relative_humidity_2m,"
17         "wind_speed_10m,weather_code",
18     }
19     r = requests.get(url, params=params, timeout=10)
20     return r.json()["current"]

```

Listing 19 pcb_bridge.py — geocoding e meteo

Formattazione del pacchetto WXC

La stringa inviata al firmware ha il formato:

WXC,City,temp_C,wind_kmh,humidity_pct,wmo_code

Lato firmware, il parsing in `sys_manager_handle_serial()` usa `indexOf` su Arduino String per trovare i cinque separatori:

```
1  int p1 = cmd.indexOf(','); // dopo "WXC"
2  int p2 = cmd.indexOf(',', p1+1); // dopo City
3  int p3 = cmd.indexOf(',', p2+1); // dopo temp
4  int p4 = cmd.indexOf(',', p3+1); // dopo wind
5  int p5 = cmd.indexOf(',', p4+1); // dopo humidity
6
7  if (p5 != -1) { // pacchetto completo
8      cmd.substring(p1+1, p2).toCharArray(w.city, 32);
9      w.temp_ext = cmd.substring(p2+1, p3).toFloat();
10     w.wind_speed = cmd.substring(p3+1, p4).toFloat();
11     w.humidity = cmd.substring(p4+1, p5).toInt();
12     w.weather_code = cmd.substring(p5+1).toInt();
13     w.valid = true;
14     sys_manager_update_weather(w);
15     Serial.println("OK_WXC");
16 }
```

Listing 20 system_manager.cpp — parsing WXC

Se uno qualsiasi dei cinque separatori manca (`p5 == -1`), il pacchetto viene scartato senza risposta. Lo script Python interpreta l'assenza di `OK_WXC` entro il timeout come errore.

6 Conclusioni e Sviluppi Futuri

Il presente lavoro ha permesso di prototipare e validare un sistema di acquisizione dati, dimostrando concretamente come l'integrazione sinergica tra un layout hardware accurato e soluzioni software ottimizzate possa colmare il gap prestazionale tipico delle piattaforme embedded a basso costo. L'impiego del SoC RP2040 non è stato limitato alle sue funzioni base, ma ne sono state sfruttate le peculiarità architettoniche (PIO, Dual-Core) per ottenere uno strumento di *data-logging* di classe industriale.

6.1 Analisi dei Risultati e Validazione Tecnica

L'architettura proposta ha permesso di raggiungere i seguenti obiettivi chiave, verificati in fase di test:

- **Integrità del Dato e Resilienza:** L'adozione del filesystem LittleFS [14] con approccio *copy-on-write* si è rivelata determinante. Durante i test di interruzione improvvisa dell'alimentazione, il sistema non ha riportato corruzioni della tabella dei descrittori, garantendo la persistenza dei log GNSS sulla Flash W25Q16JV [13].
- **Determinismo e Signal Integrity:** L'offloading del protocollo dei LED WS2812B sul sottosistema PIO ha risolto il conflitto critico tra timing dei LED e interrupt della UART. Questo ha permesso di mantenere un flusso costante di frasi NMEA [16] senza perdita di byte, processate in modo deterministico dalla libreria minmea [9], validando l'efficacia dell'approccio di *hardware offloading* e separazione dei ruoli fra cores.
- **Analisi delle Limitazioni Hardware e Diagnostica ADC:** In fase di validazione sperimentale, il sottosistema di monitoraggio termico — comprendente sia il sensore di temperatura esterno analogico sia il diodo termico interno al SoC RP2040 — non ha raggiunto l'operatività. Le sessioni di debug hardware hanno isolato la causa in un cortocircuito localizzato sulla linea di alimentazione analogica, il quale ha azzerato la tensione sul pin di riferimento dell'ADC ($V_{REF} = 0\text{ V}$). L'assenza di una tensione di riferimento stabile ha inibito il funzionamento dell'intero blocco di conversione analogico-digitale del microcontrollore, portando le letture in saturazione logica permanente. Sebbene il firmware includesse i driver di campionamento, si è reso necessario escludere i task di lettura analogica per preservare l'integrità del sistema in attesa di un ripristino elettrico della risorsa.

6.2 Riflessioni sul Design Ibrido

Il progetto ha evidenziato come il limite dei sistemi embedded moderni non sia più la potenza di calcolo bruta, ma l'efficienza nella gestione delle periferiche. La separazione dei task tra i due cores ha permesso di mantenere una latenza di risposta ai pulsanti molto bassa, nonostante il pesante carico di scrittura su memoria non volatile e il parsing continuo di stringhe (Operazione computazionalmente intensiva).

6.3 Sviluppi Futuri e Scalabilità

Nonostante l'ottimo grado di maturità raggiunto dal prototipo, sono state individuate diverse direttrici per l'evoluzione del sistema:

1. **Compressione Dati e estensione della vita utile:** Implementazione di algoritmi di compressione (come LZ4 o delta-encoding per le coordinate) per ridurre il numero di scritture fisiche sulla Flash, estendendo la vita utile del chip.

2. **Sincronizzazione Temporale Avanzata:** Utilizzo del segnale *Timepulse* (PPS) del modulo GNSS per disciplinare l'oscillatore interno dell'RP2040, permettendo data-logging con precisione temporale al microsecondo.
3. **Connettività LPWAN:** Integrazione di un ricetrasmittitore (ad es. LoRaWAN) per l'invio di pacchetti di telemetria sintetizzati wireless, trasformando il logger in un nodo IoT attivo per il monitoraggio remoto.
4. **Power Management Predittivo:** Implementazione di una logica di *Dynamic Voltage and Frequency Scaling* (DVFS) basata sullo stato di carica della batteria e sulla frequenza di acquisizione richiesta.
5. **Ripristino del Dominio Analogico e Ingegnerizzazione del Layout:** Revisione del circuito di alimentazione per eliminare il cortocircuito sulla linea analogica e isolamento del pin V_{REF} dell'RP2040 tramite l'inserimento di un riferimento di tensione dedicato e filtrato (es. un regolatore shunt di precisione a 3,3 V). Questo intervento consentirà il ripristino funzionale dell'ADC e la conseguente riattivazione dei task di telemetria termica, sia per il diodo interno al chip sia per la sonda analogica esterna.

In conclusione, il sistema sviluppato non rappresenta solo un esercizio di stile accademico, ma una solida base per lo sviluppo di unità di monitoraggio ambientale affidabile, modulare e pronta per un'eventuale ingegnerizzazione di pre-serie.

Riferimenti bibliografici

- [1] u-blox AG. *SAM-M8Q u-blox M8 GNSS antenna module Data Sheet*, 2020. Rev. R06.
- [2] Raspberry Pi Ltd. *RP2040 Datasheet: A microcontroller by Raspberry Pi*, 2023. Rev. 2.1. Disponibile online: <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>.
- [3] Eben Upton and James Adams. The rp2040: A high-performance microcontroller with programmable i/o. In *2021 IEEE Hot Chips 33 Symposium (HCS)*, pages 1–23. IEEE, 2021.
- [4] Selmir Kusi, Mattia Fait, and Alessandro Biasioli. Ppse lab firmware api documentation. <https://4137314.github.io/ppse-lab-firmware/api/index.html>, 2026.
- [5] Selmir Kusi, Mattia Fait, and Alessandro Biasioli. Ppse lab firmware homepage. <https://4137314.github.io/ppse-lab-firmware/>, 2026.
- [6] Selmir Kusi, Mattia Fait, and Alessandro Biasioli. ppse-lab-firmware: Repository sorgente. <https://github.com/4137314/ppse-lab-firmware>, 2026.
- [7] Selmir Kusi, Mattia Fait, and Alessandro Biasioli. Academic report - ppse lab firmware. https://4137314.github.io/ppse-lab-firmware/Academic_Report.pdf, 2026.
- [8] Selmir Kusi, Mattia Fait, and Alessandro Biasioli. Technical reference - ppse lab firmware. https://4137314.github.io/ppse-lab-firmware/Technical_Reference.pdf, 2026.
- [9] Kosma Moczek. minmea: a lightweight gps parser library in pure c. <https://github.com/kosma/minmea>, 2021. Accessed: 2026-01-24.
- [10] Adafruit Industries. Adafruit gfx graphics library. <https://github.com/adafruit/Adafruit-GFX-Library>, 2024. Accessed: 2026-01-24.
- [11] Adafruit Industries. Adafruit ssd1306 oled driver library for arduino. https://github.com/adafruit/Adafruit_SSD1306, 2024. Accessed: 2026-01-24.
- [12] Daniel Garcia and Mark Kriegsman. Fastled: A library for easily & efficiently controlling a wide variety of led chipsets. <https://fastled.io/>, 2024. Accessed: 2026-01-24.
- [13] Winbond Electronics. *W25Q16JV: 3V 16M-Bit Serial Flash Memory with dual/quad SPI*, 2021. Rev. I.
- [14] ARM Ltd. Littlefs: A high-integrity embedded file system for microcontrollers. <https://github.com/littlefs-project/littlefs>, 2023. Accessed: 2025-01-24.
- [15] Earle F. Philhower. *Raspberry Pi Pico Arduino core, based on the official Raspberry Pi C/C++ SDK*, 2023. Version 3.0.0.
- [16] Nmea 0183: Standard for interfacing marine electronic devices. Standard, National Marine Electronics Association (NMEA), Severna Park, MD, USA, 2008. Version 4.00.